

main.py

ShareRun

```
1 def findIntersectionValues(nums1, nums2):
2
3     set1 = set(nums1)
4     set2 = set(nums2)
5
6     answer1 = 0
7     answer2 = 0
8
9     for x in nums1:
10         if x in set2:
11             answer1 += 1
12
13     for x in nums2:
14         if x in set1:
15             answer2 += 1
16
17     return [answer1, answer2]
18
19
20 # Input
21 nums1 = [2, 3, 2]
22 nums2 = [1, 2]
23
24 print(findIntersectionValues(nums1, nums2))
```

Output

[2, 1]

=== Code Execution Successful ===

JS

TS



main.py

Share

Run

Output

```
1 def firstPalindrome(words):
2     for word in words:
3         if word == word[::-1]:
4             return word
5
6     return ""
7 words = ["abc", "car", "ada", "racecar", "cool"]
8 print(firstPalindrome(words))
```

ada

=== Code Execution Successful ===



JS

TS

```
main.py  [Icons] [Settings] [Share] [Run] [Output]

1 - def sumCounts(nums):
2
3     answer = 0
4     n = len(nums)
5
6     for i in range(n):
7
8         unique = set()
9
10        for j in range(i, n):
11
12            unique.add(nums[j])
13
14            distinct = len(unique)
15
16            answer += distinct * distinct
17
18    return answer
19
20
21 # Input
22 nums = [1, 2, 1]
23
24 print(sumCounts(nums))
```

15

--- Code Execution Successful ---

main.py

Share

Run

```
1 def countPairs(nums, k):
2
3     count = 0
4     n = len(nums)
5
6     for i in range(n):
7         for j in range(i + 1, n):
8
9             if nums[i] == nums[j] and (i * j) % k == 0:
10                 count += 1
11
12     return count
13
14
15 # Input
16 nums = [3, 1, 2, 2, 2, 1, 3]
17 k = 2
18
19 print(countPairs(nums, k))
```

Output

4

--- Code Execution Successful ---

The screenshot displays a code editor interface with a dark theme. On the left side, there is a vertical toolbar containing icons for file management, search, and other development tools. The main area is divided into two panels. The left panel shows the source code for a file named `main.py`. The code defines a function `findMax(arr)` that iterates through an array to find the maximum value. It also includes three test cases: `[1, 2, 3, 4, 5]`, `[7, 7, 7, 7, 7]`, and `[-10, 2, 3, -4, 5]`. The right panel, titled "Output", shows the results of running the code: the values 5, 7, and 5 are printed for the respective test cases, followed by a confirmation message "=== Code Execution Successful ===". At the bottom of the editor, there is a tab bar with labels for different languages: JS, TS, and GO.

```
def findMax(arr):  
    max_value = arr[0]  
    for i in range(1, len(arr)):  
        if arr[i] > max_value:  
            max_value = arr[i]  
    return max_value  
  
# Test Case 1  
print(findMax([1, 2, 3, 4, 5]))  
  
# Test Case 2  
print(findMax([7, 7, 7, 7, 7]))  
  
# Test Case 3  
print(findMax([-10, 2, 3, -4, 5]))
```

5
7
5
=== Code Execution Successful ===

main.py

Share

Run

```
1- def merge_sort(arr):
2-     if len(arr) <= 1:
3-         return arr
4-     mid = len(arr) // 2
5-     left = merge_sort(arr[:mid])
6-     right = merge_sort(arr[mid:])
7-     result = []
8-     i = 0
9-     j = 0
10-    while i < len(left) and j < len(right):
11-        if left[i] <= right[j]:
12-            result.append(left[i])
13-            i += 1
14-        else:
15-            result.append(right[j])
16-            j += 1
17-    result.extend(left[i:])
18-    result.extend(right[j:])
19-    return result
20- def findMaximum(arr):
21-     if len(arr) == 0:
22-         return None
23-     sorted_arr = merge_sort(arr)
24-     return sorted_arr[-1]
25- arr = [5, 2, 9, 1, 7]
26- print(findMaximum(arr))
```

5

7

5

Code Execution Successful

main.py

Share

Run

```
1 def uniqueElements(arr):
2
3     unique = []
4
5     seen = set()
6
7     for x in arr:
8
9         if x not in seen:
10             unique.append(x)
11             seen.add(x)
12
13     return unique
14
15
16 # Test Case 1
17 print(uniqueElements([3, 7, 3, 5, 2, 5, 9, 2]))
18
19 # Test Case 2
20 print(uniqueElements([-1, 2, -1, 3, 2, -2]))
21
22 # Test Case 3
23 print(uniqueElements([1000000, 999999, 1000000]))
```

Output

[3, 7, 5, 2, 9]
[-1, 2, 3, -2]
[1000000, 999999]

=== Code Execution Successful ===

main.py



Run

Output

```
1 def bubble_sort(arr):
2
3     n = len(arr)
4
5     for i in range(n - 1):
6
7         swapped = False
8
9         for j in range(n - i - 1):
10
11             if arr[j] > arr[j + 1]:
12
13                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
14
15                 swapped = True
16
17         if not swapped:
18             break
19
20     return arr
21
22
23 # Input
24 arr = [5, 2, 8, 1, 3]
25
26 print(bubble_sort(arr))
```

```
[1, 2, 3, 5, 8]
```

main.py

Share

Run

```
1 def binary_search(arr, key):
2     left = 0
3     right = len(arr) - 1
4     while left <= right:
5         mid = (left + right) // 2
6         if arr[mid] == key:
7             return mid
8         elif arr[mid] < key:
9             left = mid + 1
10        else:
11            right = mid - 1
12    return -1
13 arr = [3, 4, 6, -9, 10, 8, 9, 30]
14 arr.sort()
15 key = 10
16 result = binary_search(arr, key)
17 if result != -1:
18     print("Element", key, "is found at position", result + 1)
19 else:
20     print("Element", key, "is not found")
```

Output

Clear

Element 10 is found at position 7

=== Code Execution Successful ===

Python Online Compiler

PRV14491 CRICOS 043741

In the world*

Ranked equal 82, 2020 QS World University Rankings.

LEARNIT MYNG

main.py

Share

Run

```
1 def merge_sort(arr):
2     if len(arr) <= 1:
3         return arr
4     mid = len(arr) // 2
5     left = merge_sort(arr[:mid])
6     right = merge_sort(arr[mid:])
7     result = []
8     i = 0
9     j = 0
10    while i < len(left) and j < len(right):
11        if left[i] <= right[j]:
12            result.append(left[i])
13            i += 1
14        else:
15            result.append(right[j])
16            j += 1
17    result.extend(left[i:])
18    result.extend(right[j:])
19    return result
20 nums = [5, 2, 8, 1, 3]
21
22 print(merge_sort(nums))
```

Output

[1, 2, 3, 5, 8]

=== Code Execution Successful ===

main.py

Run

Share

Clear

```
1- def findPaths(m, n, N, i, j):
2-     MOD = 1000000007
3-     dp = [[0] * n for _ in range(m)]
4-     dp[i][j] = 1
5-     answer = 0
6-     directions = [
7-         (1, 0),
8-         (-1, 0),
9-         (0, 1),
10-        (0, -1)
11-    ]
12-
13-    for step in range(N):
14-
15-        new_dp = [[0] * n for _ in range(m)]
16-        for r in range(m):
17-            for c in range(n):
18-
19-                if dp[r][c] == 0:
20-                    continue
21-                for dr, dc in directions:
22-                    nr = r + dr
23-                    nc = c + dc
24-                    if nr < 0 or nr >= m or nc < 0 or nc >= n:
25-                        answer = (answer + dp[r][c]) % MOD
26-                    else:
27-                        new_dp[nr][nc] = (
28-                            new_dp[nr][nc] + dp[r][c]
29-                        ) % MOD
30-        dp = new_dp
31-    return answer
32- print(findPaths(2, 2, 2, 0, 0))
```

Output

6

--- Code Execution Successful ---

main.py

Share

Run

```
1 def rob_linear(nums, start, end):
2     prev2 = 0
3     prev1 = 0
4     for i in range(start, end + 1):
5         current = max(
6             prev1,
7             prev2 + nums[i]
8         )
9         prev2 = prev1
10        prev1 = current
11    return prev1
12 def rob(nums):
13     n = len(nums)
14     if n == 1:
15         return nums[0]
16     case1 = rob_linear(nums, 0, n - 2)
17     case2 = rob_linear(nums, 1, n - 1)
18     return max(case1, case2)
19 print(rob([2, 3, 2]))
20 print(rob([1, 2, 3, 1]))
```

Output

3
4

=== Code Execution Successful ===

main.py

Share

Run

```
1- def climbStairs(n):
2
3-     if n <= 2:
4         return n
5
6     a = 1
7     b = 2
8
9-     for i in range(3, n + 1):
10
11         c = a + b
12
13         a = b
14         b = c
15
16     return b
17
18
19 print(climbStairs(4))
20 print(climbStairs(3))
```

Output

5
3

=== Code Execution Successful ===

Python

JS

TS

GO

mcin.py

1 def uniquePaths(m, n):

2

3 dp = [1] * n

4

5 for i in range(1, m):

6

7 for j in range(1, n):

8

9 dp[j] = dp[j] + dp[j - 1]

10

11 return dp[n - 1]

12

13

14 print(uniquePaths(7, 3))

15 print(uniquePaths(3, 2))

Full Screen

Dark Mode

Share

Run

Output

28

3

=== Code Execution Successful ===



main.py

 Share

 Run

Output

```
1 def largeGroupPositions(s):
2
3     result = []
4
5     start = 0
6
7     for i in range(1, len(s) + 1):
8
9         if i == len(s) or s[i] != s[start]:
10
11             if i - start >= 3:
12
13                 result.append([start, i - 1])
14
15                 start = i
16
17     return result
18
19
20 # Input
21 s = "abbxxxxzzy"
22
23 print(largeGroupPositions(s))
```

[[3, 6]]

=== Code Execution Successful ===

The screenshot displays a code editor interface with a dark theme. The left sidebar contains icons for file management, search, and other IDE features. The main editor area shows a Python script named `main.py`. The script defines a function `gameOfLife(board)` that simulates one step of Conway's Game of Life. It takes a 3x3 grid as input and returns the next state. The logic involves counting live neighbors for each cell and updating its state based on the rules of the game.

```
1- def gameOfLife(board):  
2-     m = len(board)  
3-     n = len(board[0])  
4-     original = [row[:] for row in board]  
5-     directions = [  
6-         (-1, -1), (-1, 0), (-1, 1),  
7-         (0, -1),          (0, 1),  
8-         (1, -1),   (1, 0),   (1, 1)  
9-     ]  
10-    for i in range(m):  
11-        for j in range(n):  
12-            live = 0  
13-            for di, dj in directions:  
14-                ni = i + di  
15-                nj = j + dj  
16-                if (0 <= ni < m and  
17-                    0 <= nj < n and  
18-                    original[ni][nj] == 1):  
19-                    live += 1  
20-            if original[i][j] == 1:  
21-                if live < 2 or live > 3:  
22-                    board[i][j] = 0  
23-            else:  
24-                if live == 3:  
25-                    board[i][j] = 1  
26-    board = [  
27-        [0, 1, 0],  
28-        [0, 0, 1],  
29-        [1, 1, 1],  
30-        [0, 0, 0]  
31-    ]  
32-    gameOfLife(board)  
33-    for row in board:  
34-        print(row)
```

The right sidebar shows the output of the program, which consists of four rows representing the final state of the board after multiple iterations:

```
[0, 0, 0]  
[1, 0, 1]  
[0, 1, 1]  
[0, 1, 0]
```

Below the output, there is a message indicating successful execution: `=== Code Execution Successful ===`.

